

Ahmad Azroun

Renewable Energy Manager | IoT & Smart Energy Systems Developer

ESP32 Smart Room Controller V4 - Deutsche Dokumentation

Dieses Projekt basiert auf einem ESP32-Mikrocontroller und kombiniert Sensorik, intelligente Beleuchtungssteuerung, OLED-Echtzeitanzeige, Web-Dashboard und MQTT-Kommunikation. Das Ziel ist die Entwicklung eines professionellen Smart-Room-Controllers, der Umgebungslicht erkennt, Bewegung überwacht und die RGB-Beleuchtung automatisch oder manuell steuert. Durch MQTT kann das System sowohl über einen Computer als auch über ein Smartphone aus der Ferne überwacht und gesteuert werden.

Systemuebersicht

Komponente	Funktion
ESP32	Zentrale Steuerung, WLAN-Verbindung und MQTT-Kommunikation
PIR Sensor	Erkennt Bewegung im Raum
LDR Sensor	Misst die Umgebungshelligkeit
RGB LED	Zeigt den aktuellen Betriebszustand farblich an
OLED Display	Zeigt Live-Daten wie Modus, Lichtwert, Bewegung und RGB-Status
MQTT Broker	Ermöglicht Fernüberwachung und Fernsteuerung ueber Internet

Das System arbeitet in drei Hauptmodi: AUTO, MANUAL_ON und MANUAL_OFF. Im AUTO-Modus entscheidet das System anhand von Licht- und Bewegungsdaten automatisch, ob die Beleuchtung ausgeschaltet, gedimmt blau oder weiß aktiviert wird. Im manuellen Betrieb kann der Benutzer ueber MQTT gezielt den gruenen oder roten LED-Zustand ausloesen.

Systembilder und Live-Demonstration - Computersteuerung

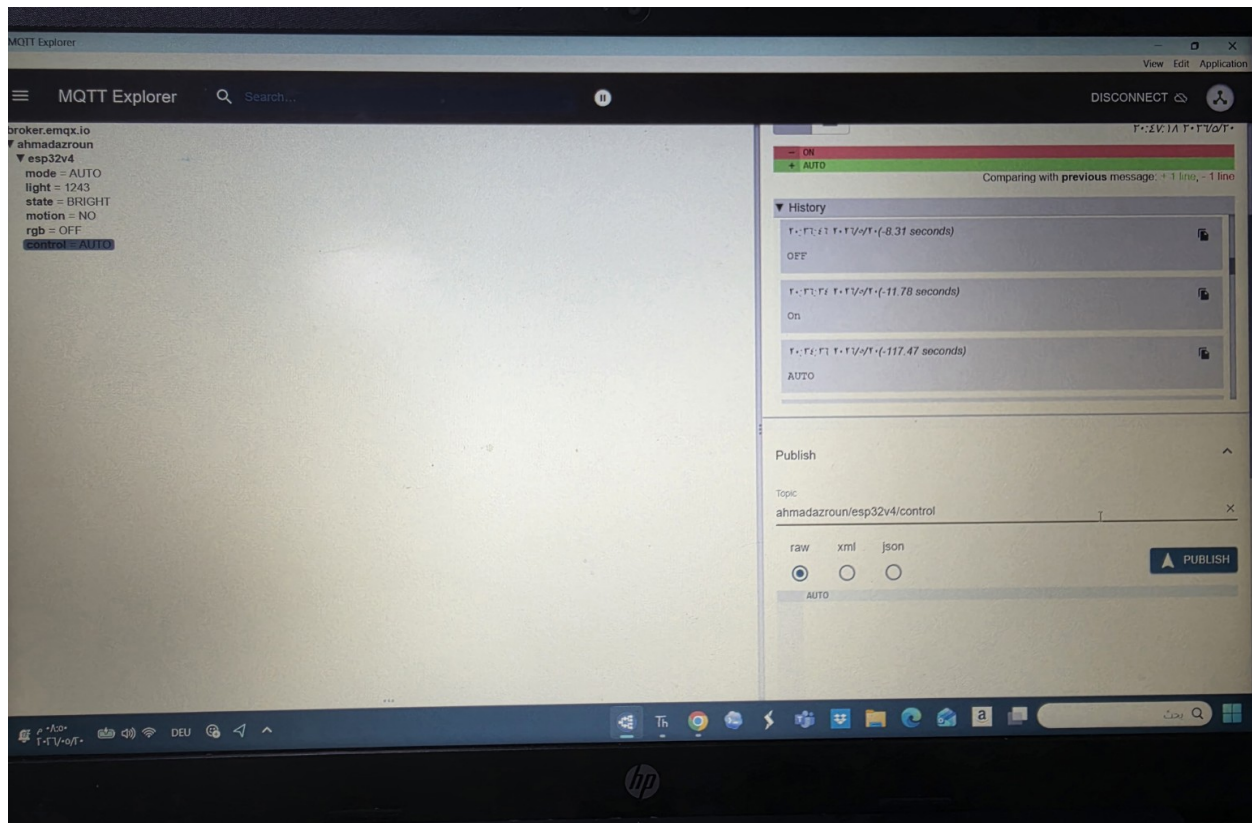


Abbildung 1: MQTT Explorer Desktop Interface

MQTT Explorer Desktop-Oberfläche mit verbundenem Broker, Live-Daten und Steuer-Topic.

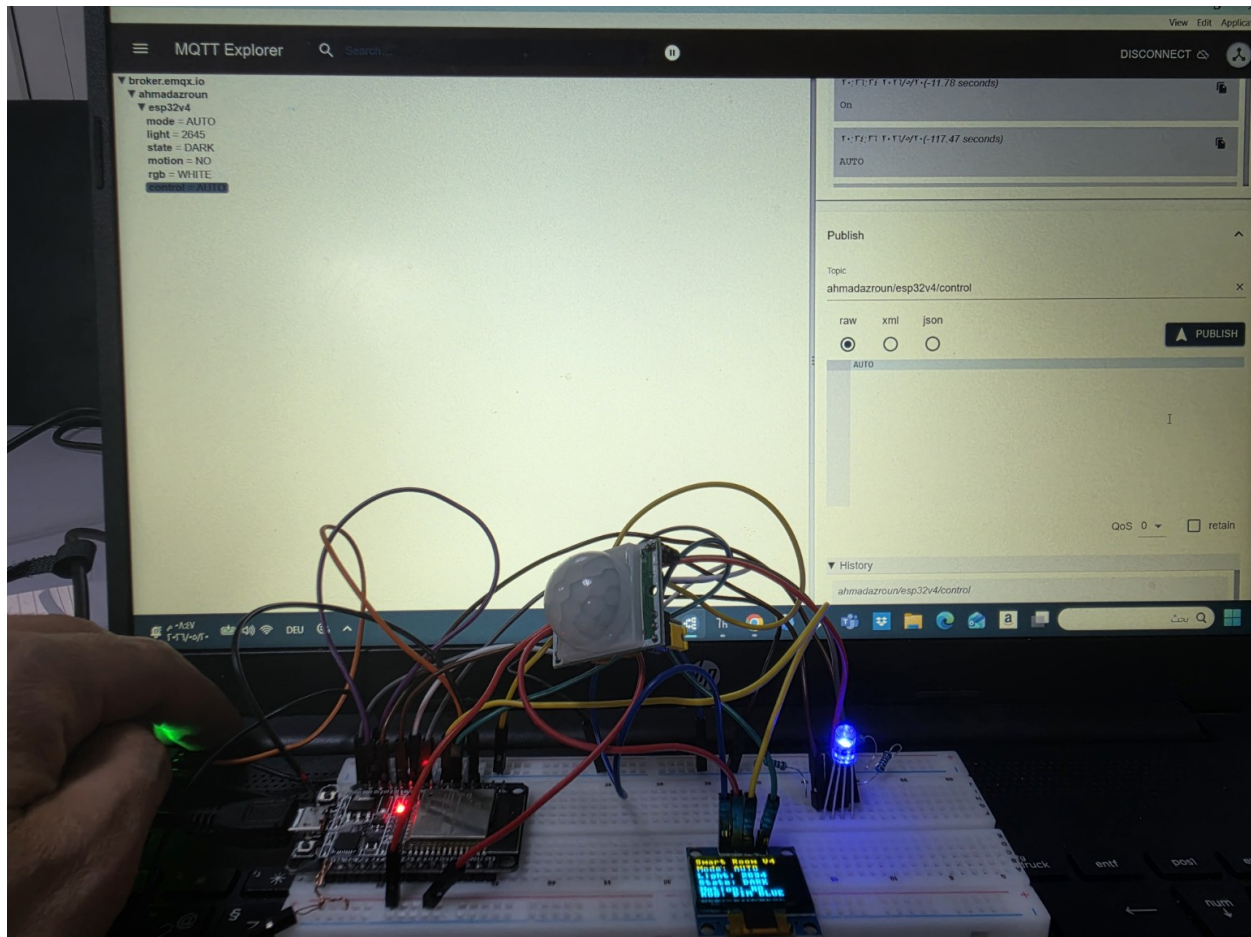


Abbildung 2: AUTO Mode - Dark Environment without Motion - Dim Blue LED
AUTO-Modus bei dunkler Umgebung ohne erkannte Bewegung: Die RGB-LED leuchtet gedimmt blau als Ambientelicht.

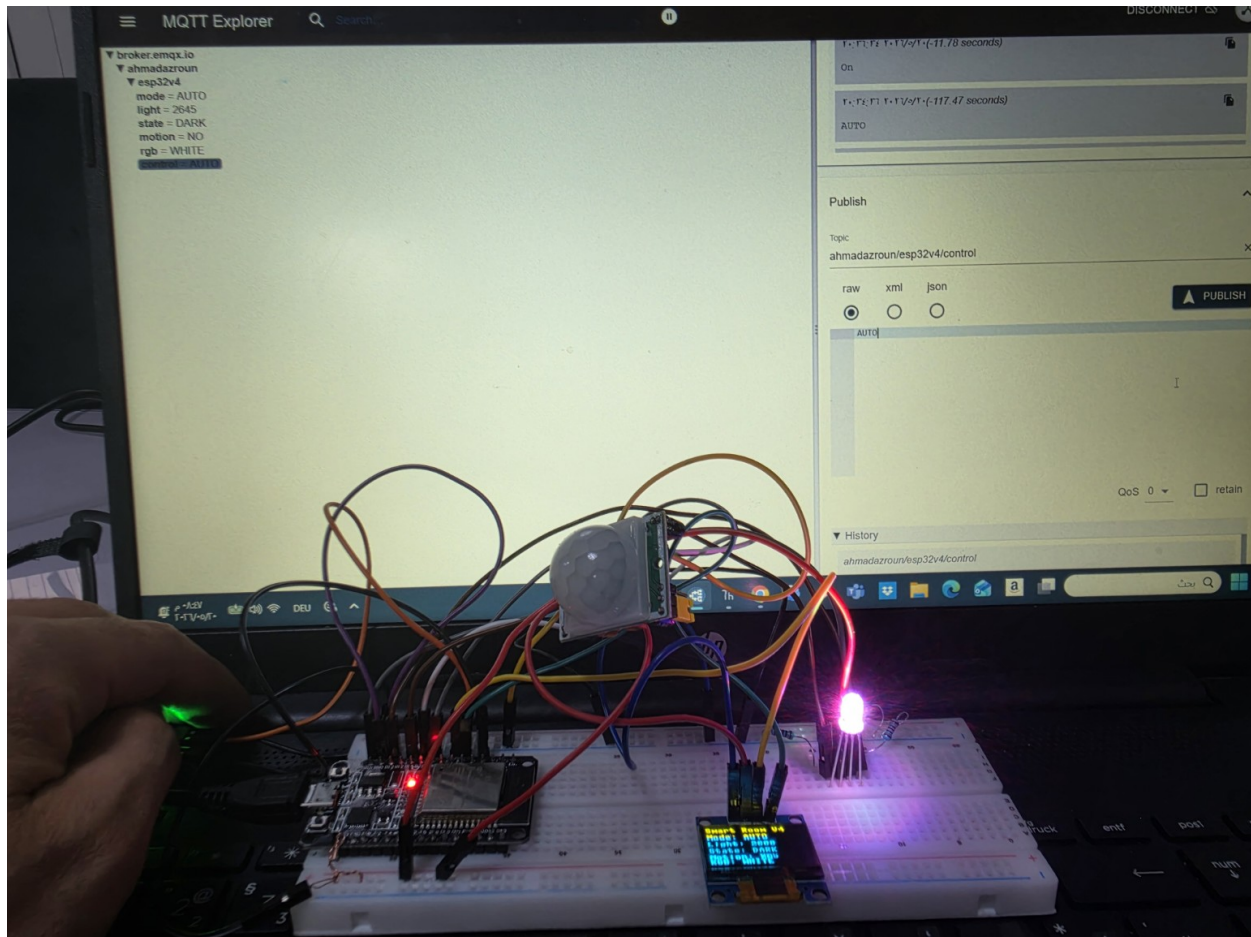


Abbildung 3: AUTO Mode - Dark Environment with Motion - White LED
AUTO-Modus bei dunkler Umgebung mit erkannter Bewegung: Die RGB-LED schaltet automatisch auf weißes Licht.

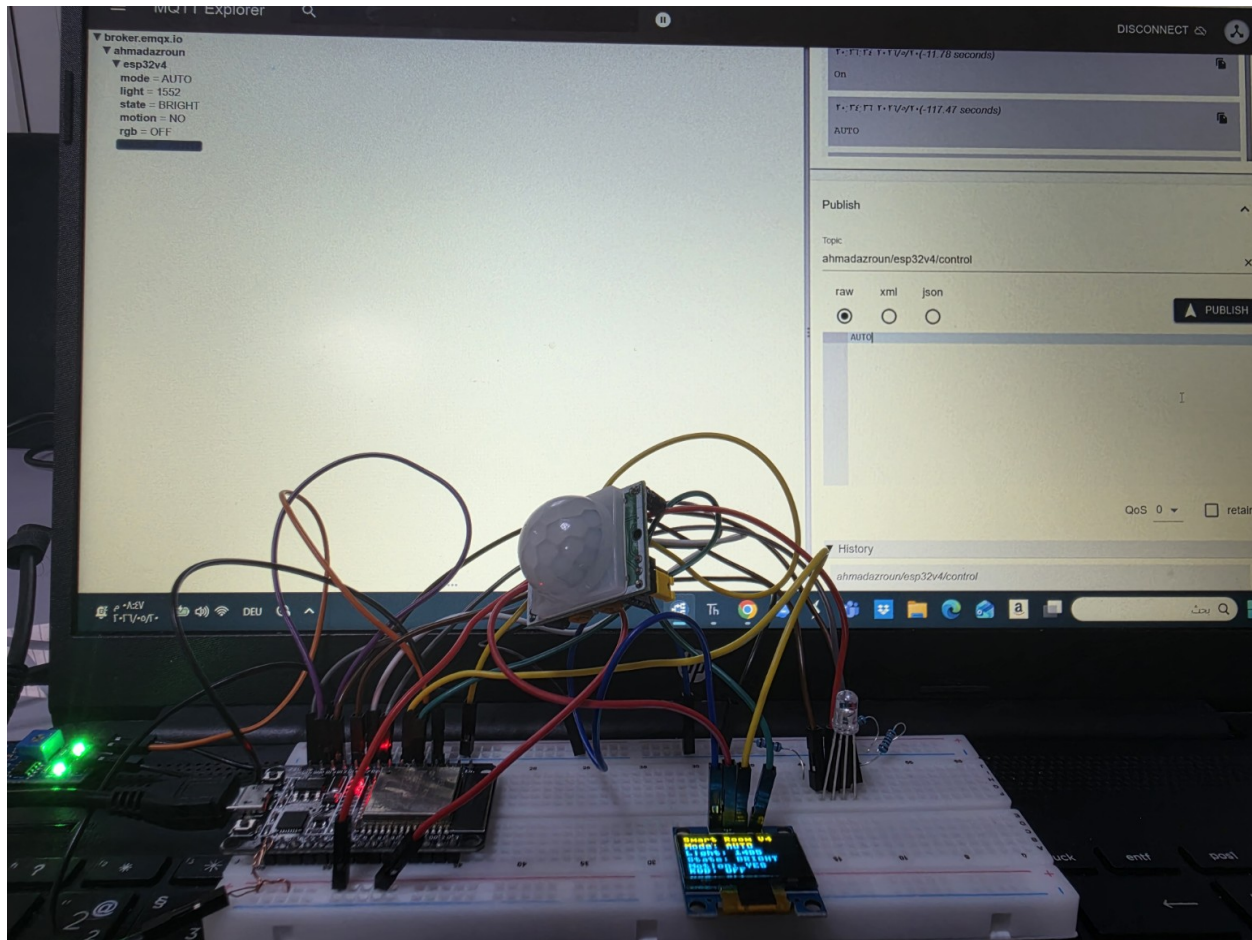


Abbildung 4: AUTO Mode - Bright External Light - LED OFF
AUTO-Modus bei ausreichender externer Beleuchtung: Das System deaktiviert die RGB-LED zur Energieeinsparung.

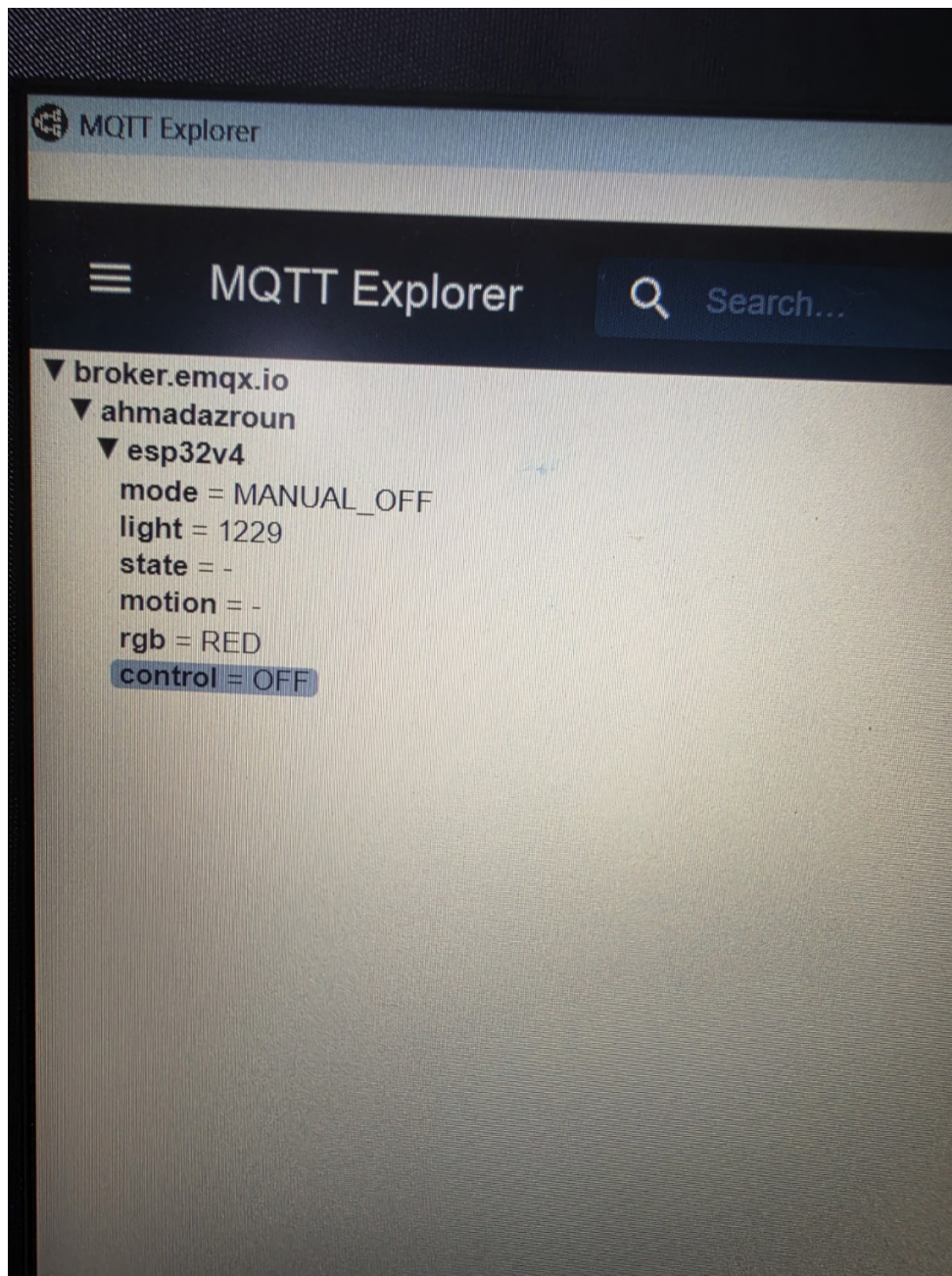


Abbildung 5: MQTT Computer Control - MANUAL OFF - Red LED
Manuelle OFF-Steuerung über MQTT Explorer am Computer: Das System aktiviert MANUAL_OFF und die RGB-LED leuchtet rot.

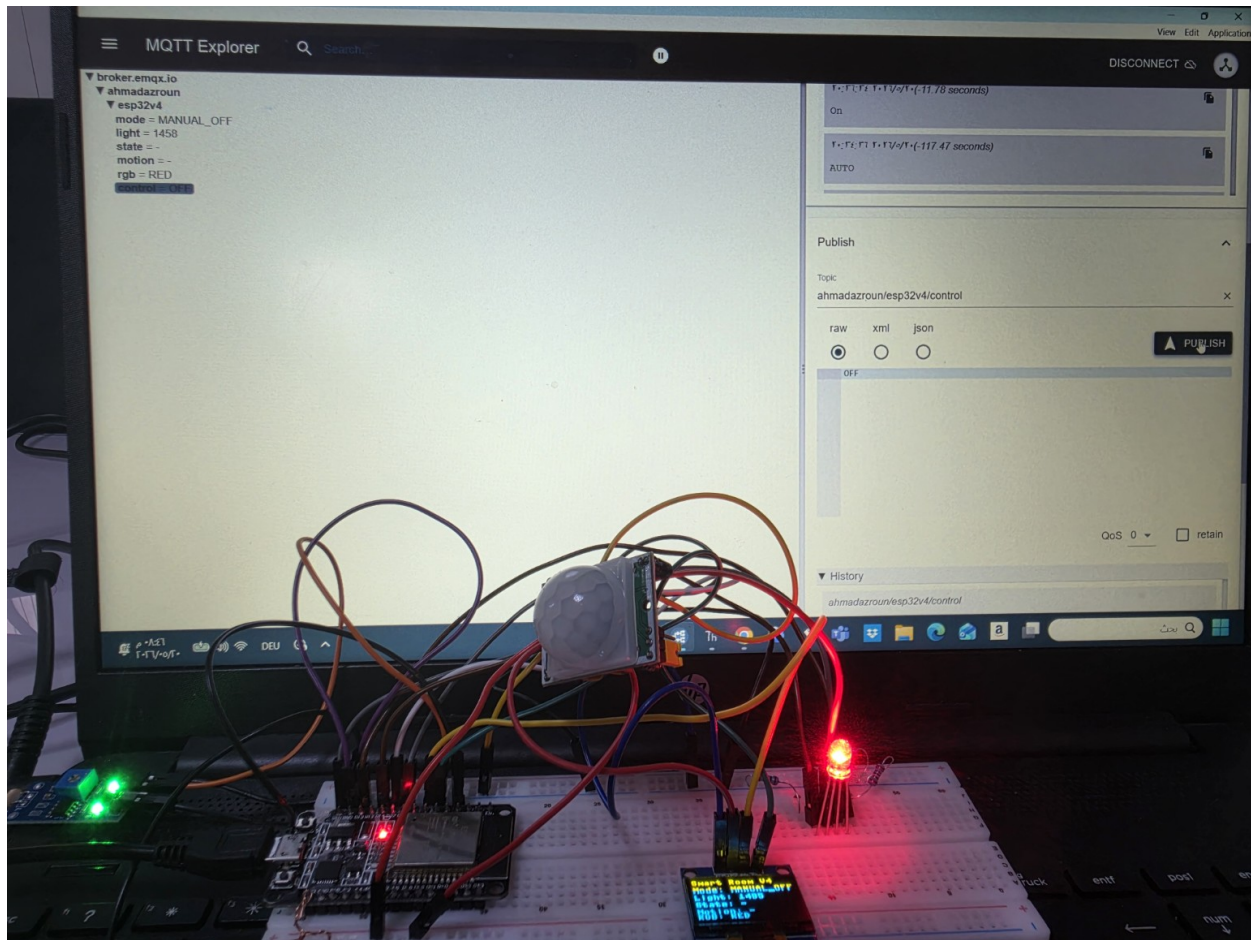


Abbildung 6: MQTT Computer Control - Red LED Status
Der rote LED-Zustand bestätigt die erfolgreiche OFF-Steuerung über das MQTT-Control-Topic.

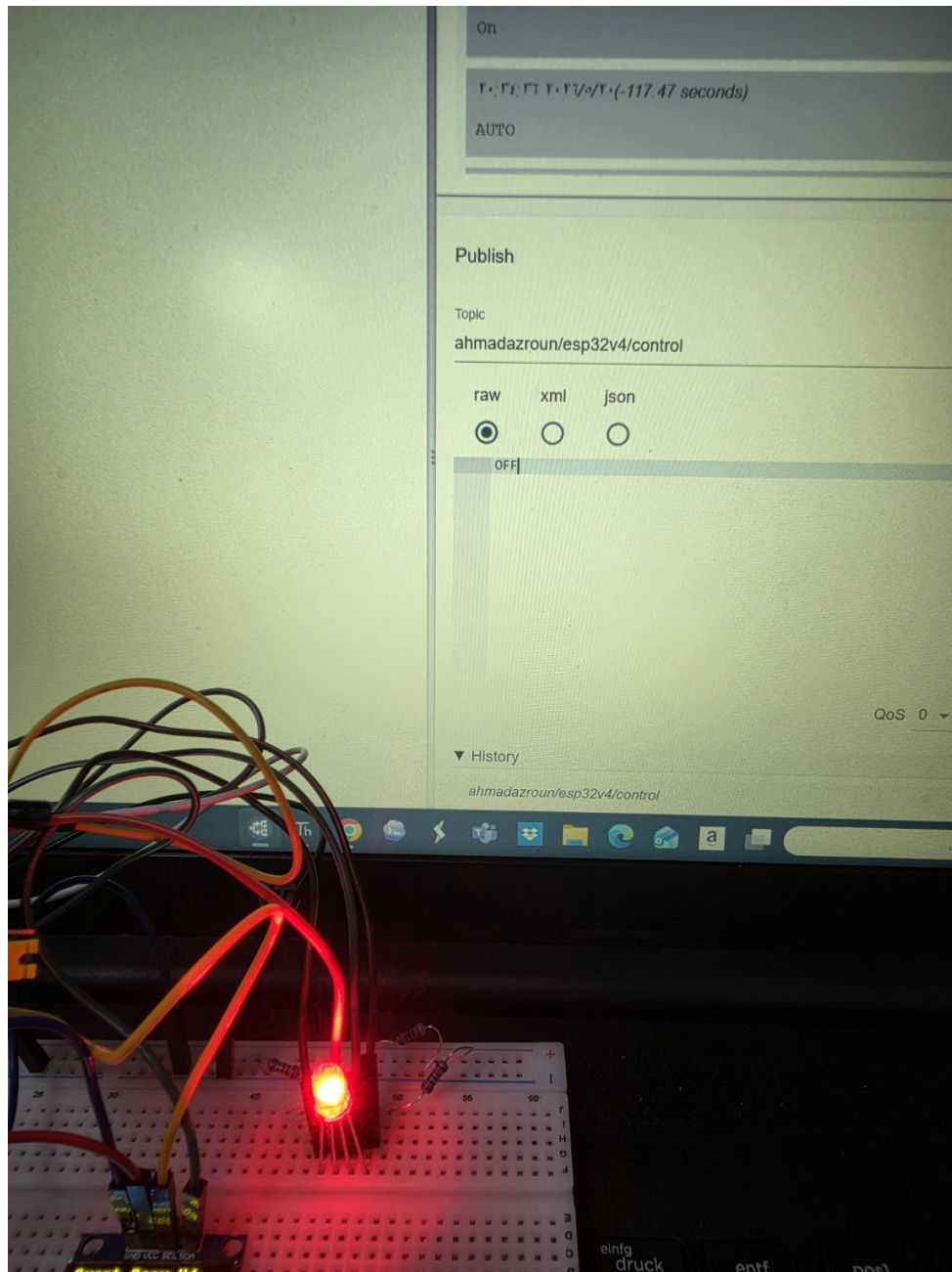


Abbildung 7: MQTT Computer Control - OFF Command Detail
Detailansicht der gesendeten OFF-Nachricht im MQTT-Publish-Bereich.

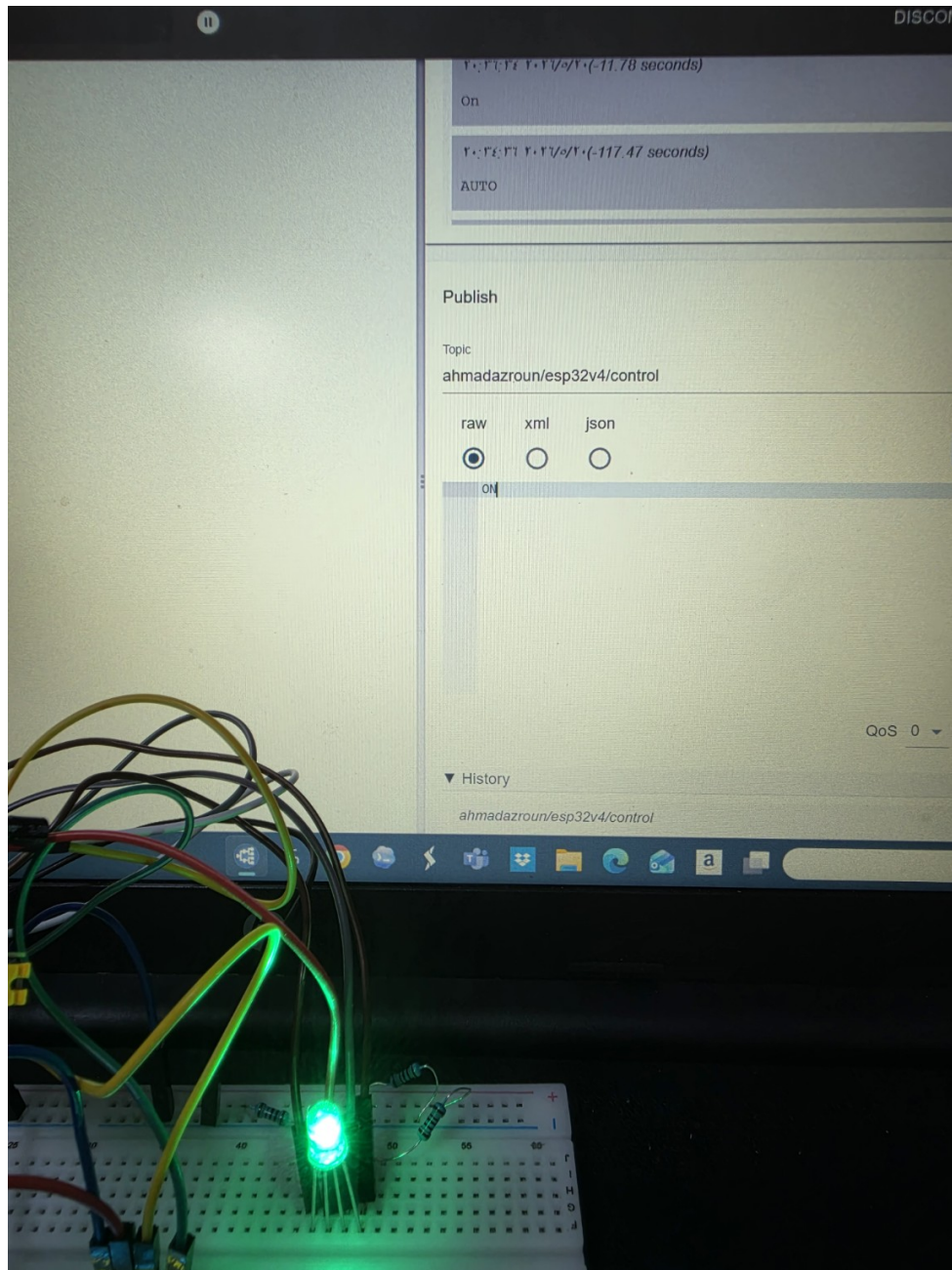


Abbildung 8: MQTT Computer Control - MANUAL ON - Green LED
Manuelle ON-Steuerung über MQTT Explorer am Computer: Die RGB-LED leuchtet grün.

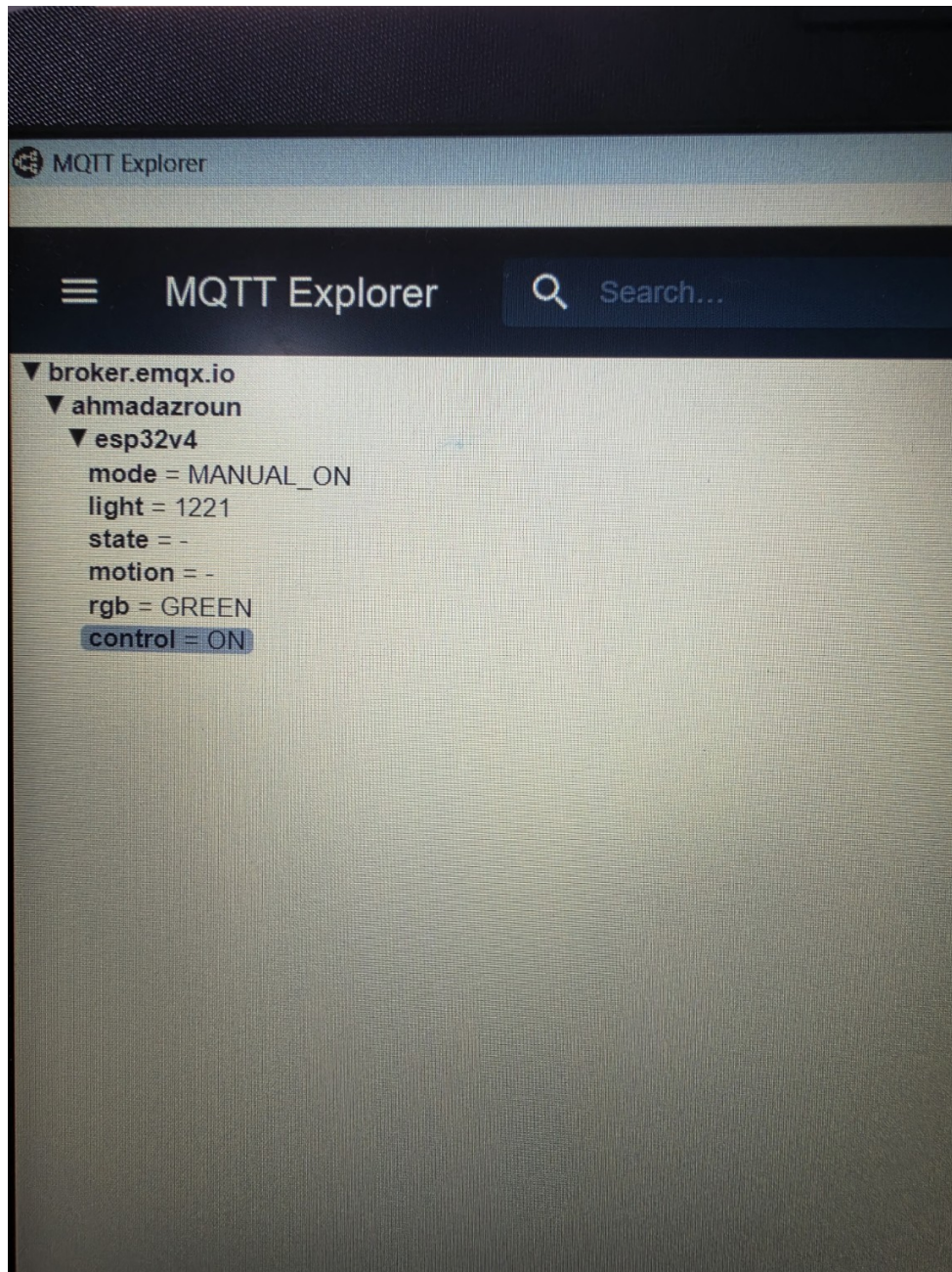


Abbildung 9: MQTT Computer Monitoring - GREEN Status
MQTT Explorer zeigt MANUAL_ON, RGB GREEN und aktuelle Sensordaten in Echtzeit.

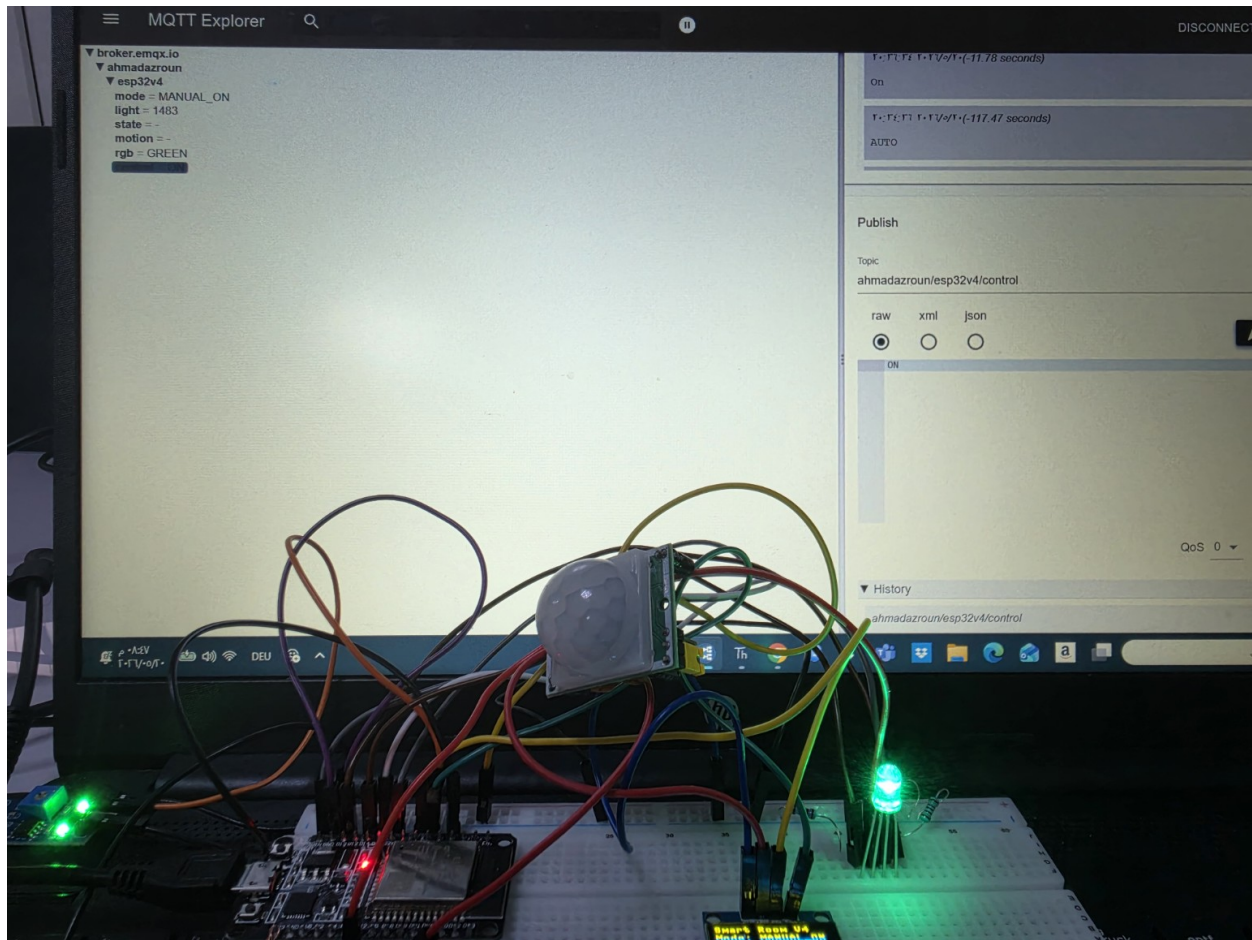


Abbildung 10: MQTT Computer Control - Green LED with System
Gesamtansicht des Projekts im MANUAL_ON-Modus mit grünem LED-Status.

Systembilder und Live-Demonstration - Smartphone Dashboard



Abbildung 11: Mobile Dashboard - AUTO Mode - Bright Light - LED OFF
Mobile MQTT-Dashboard im AUTO-Modus bei externer Beleuchtung: Die LED bleibt ausgeschaltet.



Abbildung 12: Mobile Dashboard - AUTO Mode - White LED with Motion
Mobile Steuerung und Überwachung im AUTO-Modus: Bei Bewegung in dunkler Umgebung leuchtet die LED weiß.



Abbildung 13: Mobile Dashboard - AUTO Mode - Dim Blue without Motion
 Mobile Dashboard im AUTO-Modus: Bei Dunkelheit ohne Bewegung leuchtet die LED gedimmt blau.

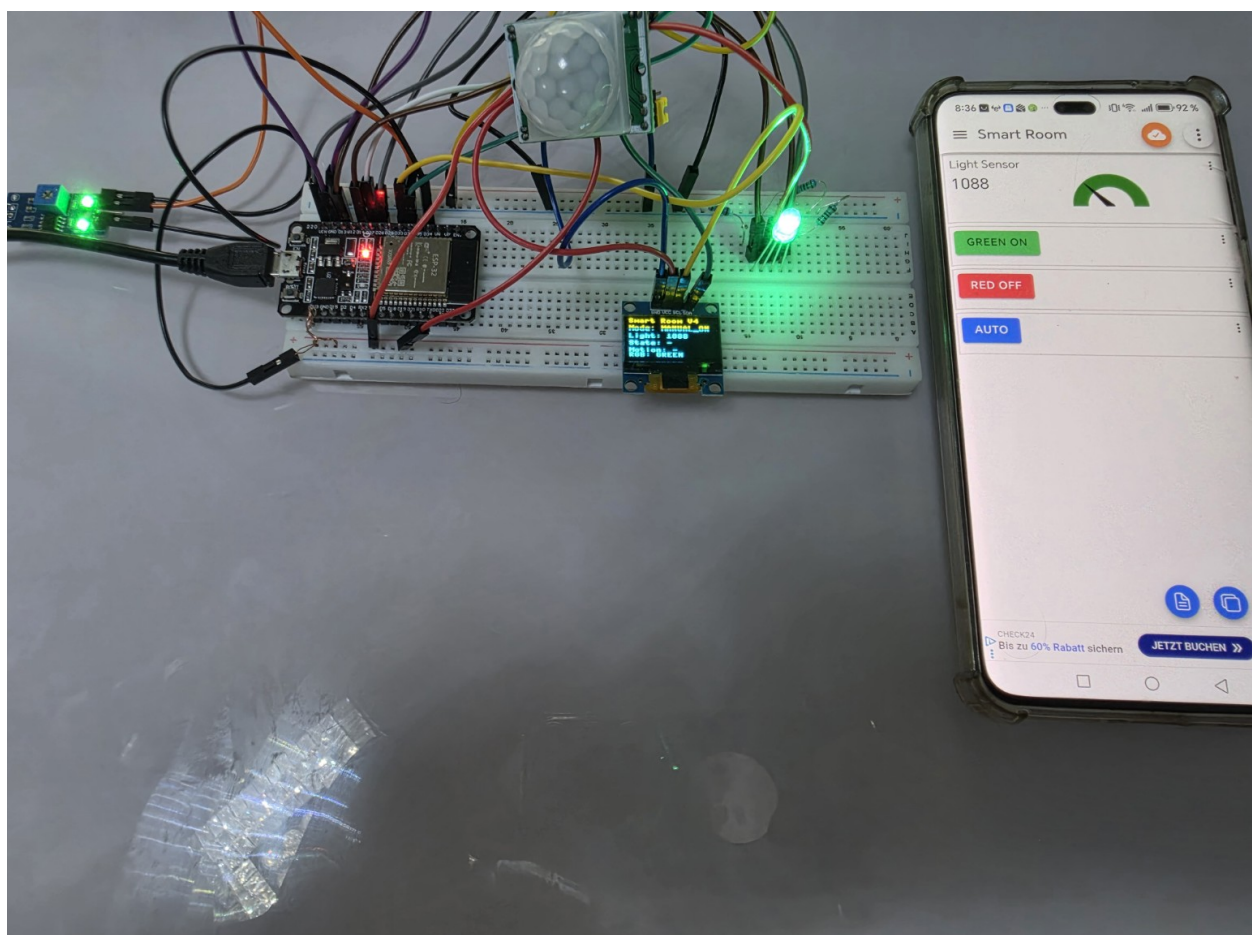


Abbildung 14: Mobile Dashboard - MANUAL ON - Green LED
Manuelle ON-Steuerung über das Smartphone: Die RGB-LED leuchtet grün.

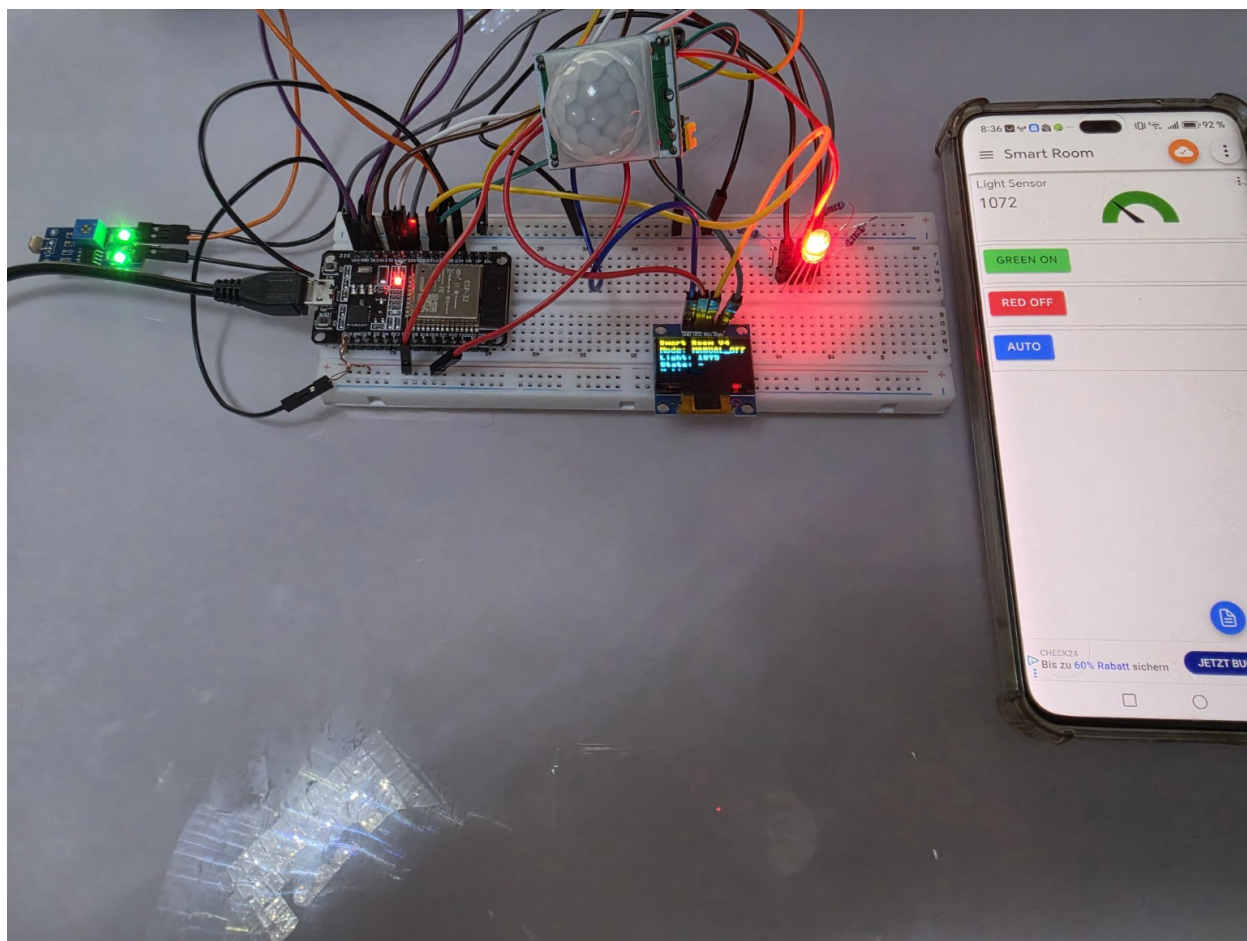


Abbildung 15: Mobile Dashboard - MANUAL OFF - Red LED
Manuelle OFF-Steuerung über das Smartphone: Die RGB-LED leuchtet rot.

Praktische Anwendungen

Anwendungsbereich	Nutzen
Smart Home	Automatische und manuelle Raumbeleuchtung mit Fernsteuerung
Energieeffizienz	Beleuchtung wird bei ausreichendem Tageslicht automatisch deaktiviert
Smart Building	Grundlage fuer intelligente Raeume, Labore, Bueros und Schulungsraeume
Sicherheit	Bewegungserkennung kann als Basis fuer Nachtlicht oder Alarmfunktionen dienen
IoT Lernen	Praktische Verbindung von Embedded Systems, Sensorik, MQTT und Dashboards

ESP32 Smart Room Controller V4 - English Documentation

The ESP32 Smart Room Controller V4 is a complete IoT-based smart lighting and monitoring system. It integrates an ESP32 microcontroller, PIR motion sensor, LDR light sensor, common-anode RGB LED, OLED display, local web dashboard, and MQTT communication. The system supports real-time monitoring and remote control from both a computer using MQTT Explorer and a smartphone using a mobile MQTT dashboard.

Operating Concept

In AUTO mode, the controller evaluates ambient light and motion status. If the environment is bright, the RGB LED remains OFF to save energy. If the environment is dark and no motion is detected, the system activates a dim blue ambient light. If motion is detected in darkness, the RGB LED changes to white light. In manual modes, the user can send MQTT commands: ON activates the green LED, OFF activates the red LED, and AUTO returns the system to automatic control.

MQTT Topic	Purpose
ahmadazroun/esp32v4/light	Publishes the live LDR light sensor value
ahmadazroun/esp32v4/motion	Publishes the PIR motion state
ahmadazroun/esp32v4/rgb	Publishes the current RGB LED status
ahmadazroun/esp32v4/mode	Publishes the current operating mode
ahmadazroun/esp32v4/control	Receives commands: ON, OFF, AUTO

Practical Applications

This prototype can be used as a foundation for smart home automation, energy-efficient lighting systems, occupancy-based room control, intelligent offices and laboratories, IoT training platforms, and smart energy demonstration systems. It shows how embedded hardware, sensor data, MQTT communication, and dashboards can be combined into a real practical IoT architecture.

Image Index and Explanation

Figure	Name	Explanation
1	MQTT Explorer Desktop Interface	Desktop MQTT Explorer connected to broker showing live data.
2	AUTO Dark without Motion	RGB LED lights dim blue in darkness without movement.
3	AUTO Dark with Motion	RGB LED switches to white when motion is detected.
4	AUTO Bright Environment	RGB LED remains OFF to save energy.
5	MQTT MANUAL OFF Control	MQTT command activates MANUAL_OFF mode.
6	MQTT OFF Status	RGB LED turns red in OFF mode.
7	MQTT OFF Publish	OFF command sent through MQTT Explorer.
8	MQTT MANUAL ON Control	MQTT command activates MANUAL_ON mode.
9	MQTT GREEN Status	MQTT Explorer displays live monitoring values.
10	MQTT GREEN Monitoring	Complete system view in MANUAL_ON mode.
11	Mobile AUTO Bright	Dashboard shows AUTO mode with LED OFF.
12	Mobile AUTO White	LED turns white during motion in darkness.
13	Mobile AUTO Dim Blue	LED turns dim blue without motion.
14	Mobile MANUAL ON	Smartphone activates green LED.
15	Mobile MANUAL OFF	Smartphone activates red LED.

Complete MicroPython Source Code

The following code represents the full logic used for the ESP32 Smart Room Controller V4. The WiFi password is intentionally replaced with a placeholder for security.

```
from machine import Pin, ADC, PWM, I2C, reset
import ssd1306
import network
import socket
import time
from umqtt.simple import MQTTClient

# WiFi
SSID = "FRITZ!Box 6670 TT"
PASSWORD = "YOUR_WIFI_PASSWORD"

# MQTT
MQTT_BROKER = "broker.emqx.io"
MQTT_CLIENT_ID = "ESP32_Ahmad_SmartRoom_V4"

MQTT_TOPIC_MODE = b"ahmadazroun/esp32v4/mode"
MQTT_TOPIC_LIGHT = b"ahmadazroun/esp32v4/light"
MQTT_TOPIC_STATE = b"ahmadazroun/esp32v4/state"
MQTT_TOPIC_MOTION = b"ahmadazroun/esp32v4/motion"
MQTT_TOPIC_RGB = b"ahmadazroun/esp32v4/rgb"
MQTT_TOPIC_CONTROL = b"ahmadazroun/esp32v4/control"

# RGB LED - Common Anode
red = PWM(Pin(14))
green = PWM(Pin(26))
blue = PWM(Pin(13))

red.freq(1000)
green.freq(1000)
blue.freq(1000)

red.duty(1023)
green.duty(1023)
blue.duty(1023)

# Sensors
light_sensor = ADC(Pin(34))
light_sensor atten(ADC.ATTN_11DB)
light_sensor.width(ADC.WIDTH_12BIT)

motion_sensor = Pin(27, Pin.IN)

# OLED
i2c = I2C(0, scl=Pin(33), sda=Pin(32), freq=100000)
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

# System Variables
system_mode = "AUTO"
motion_timeout = 5
last_motion_time = 0

startup_time = time.time()
pir_warmup_time = 25
pir_initialized = False

current_color = (0, 0, 0)

light_value = 0
light_state = "-"
motion_text = "-"
rgb_status = "OFF"

last_mqtt_publish = 0
mqtt_publish_interval = 5

# RGB Functions
def set_color(r, g, b):
    red.duty(1023 - r)
    green.duty(1023 - g)
    blue.duty(1023 - b)

def fade_color(start, end, steps=10, delay=0.003):
    for i in range(steps + 1):
        r = int(start[0] + (end[0] - start[0]) * i / steps)
        g = int(start[1] + (end[1] - start[1]) * i / steps)
        b = int(start[2] + (end[2] - start[2]) * i / steps)
        set_color(r, g, b)
        time.sleep(delay)

def go_to_color(target):
    global current_color
    if current_color != target:
        fade_color(current_color, target)
        current_color = target

# OLED Update
def update_oled():
    oled.fill(0)
    oled.text("Smart Room V4", 0, 0)
    oled.text("Mode: " + system_mode, 0, 12)
    oled.text("Light: " + str(light_value), 0, 24)
    oled.text("State: " + light_state, 0, 36)
    oled.text("Motion: " + motion_text, 0, 48)
    oled.text("RGB: " + rgb_status, 0, 56)
    oled.show()

# MQTT Callback
def mqtt_callback(topic, msg):
    global system_mode
    command = msg.decode().strip().upper()
    print("MQTT Command:", command)
    if command == "AUTO":
```

```

        system_mode = "AUTO"
        print("MQTT -> AUTO MODE")
    elif command == "ON":
        system_mode = "MANUAL_ON"
        print("MQTT -> MANUAL ON")
    elif command == "OFF":
        system_mode = "MANUAL_OFF"
        print("MQTT -> MANUAL OFF")

# Start Screen
update_oled()

# WiFi Connect
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect(SSID, PASSWORD)

print("Connecting to WiFi...")

while not wifi.isconnected():
    red.duty(1023)
    green.duty(1023)
    blue.duty(1023)
    oled.fill(0)
    oled.text("Smart Room V4", 0, 0)
    oled.text("WiFi Connecting", 0, 24)
    oled.text("RGB: OFF", 0, 40)
    oled.show()
    print("Connecting...")
    time.sleep(1)

ip = wifi.ifconfig()[0]
print("WiFi Connected")
print("ESP32 IP:", ip)

# MQTT Connect
mqtt_client = MQTTClient(MQTT_CLIENT_ID, MQTT_BROKER)
try:
    mqtt_client.set_callback(mqtt_callback)
    mqtt_client.connect()
    mqtt_client.subscribe(MQTT_TOPIC_CONTROL)
    print("MQTT Connected")
    print("Subscribed to:", MQTT_TOPIC_CONTROL)
except OSError:
    mqtt_client = None
    print("MQTT Connection Failed")

# Web Page
def webpage():
    html = f"""
<html>
<head>
<title>ESP32 Smart Room Controller V4</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body {{ background: #111827; color: white; font-family: Arial; text-align: center; padding-top: 35px; }}
.box {{ background: #1f2937; width: 85%; max-width: 650px; margin: auto; padding: 30px; border-radius: 20px; }}
h1 {{ font-size: 40px; color: #38bdf8; }}
h2 {{ font-size: 26px; }}
button {{ width: 260px; height: 60px; border: none; border-radius: 15px; margin: 10px; font-size: 22px; font-weight: bold; }}
.auto {{ background: #38bdf8; }}
.on {{ background: #22c55e; }}
.off {{ background: #ef4444; color: white; }}
.restart {{ background: #f59e0b; }}
.footer {{ position: fixed; left: 20px; bottom: 10px; font-size: 22px; color: #cbd5e1; text-align: left; }}
</style>
</head>
<body>
<div class="box">
<h1>ESP32 Smart Room Controller V4</h1>
<h2>Mode: {system_mode}</h2>
<h2>Light: {light_value}</h2>
<h2>State: {light_state}</h2>
<h2>Motion: {motion_text}</h2>
<h2>RGB: {rgb_status}</h2>
<a href="/auto"><button class="auto">AUTO MODE</button></a><br>
<a href="/on"><button class="on">MANUAL ON</button></a><br>
<a href="/off"><button class="off">MANUAL OFF</button></a><br>
<a href="/restart"><button class="restart">RESTART ESP32</button></a>
</div>
<div class="footer">Ahmad Azroun<br>Renewable Energy Manager<br>IoT & Smart Energy Systems Developer</div>
</body>
</html>
"""
    return html

# Web Server
addr = socket.getaddrinfo("0.0.0.0", 80)[0][-1]
server = socket.socket()
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(addr)
server.listen(1)
server.settimeout(0.02)
print("Web Server Running")
print("Open: http://" + ip)
update_oled()

# Main Loop
while True:
    # MQTT Check Commands
    if mqtt_client is not None:
        try:
            mqtt_client.check_msg()
        except OSError:
            print("MQTT Check Failed")

    # Web Request
    try:
        client, addr = server.accept()
        client.settimeout(0.2)
        try:
            request = client.recv(512).decode()
            first_line = request.split("\r\n")[0]

```

```

        print("Request:", first_line)
        if "GET /auto " in first_line:
            system_mode = "AUTO"
        elif "GET /on " in first_line:
            system_mode = "MANUAL_ON"
        elif "GET /off " in first_line:
            system_mode = "MANUAL_OFF"
        elif "GET /restart " in first_line:
            client.send("HTTP/1.1 200 OK\r\n")
            client.send("Content-Type: text/html\r\n")
            client.send("Connection: close\r\n\r\n")
            client.send("<h1>Restarting ESP32...</h1>")
            client.close()
            time.sleep(1)
            reset()
            response = webpage()
            client.send("HTTP/1.1 200 OK\r\n")
            client.send("Content-Type: text/html\r\n")
            client.send("Connection: close\r\n\r\n")
            client.sendall(response)
    except OSError:
        print("Client disconnected safely")
    finally:
        client.close()
except OSError:
    pass

# Read Sensors
light_value = light_sensor.read()
raw_motion = motion_sensor.value()
pir_ready = time.time() - startup_time > pir_warmup_time

# AUTO MODE
if system_mode == "AUTO":
    if not pir_ready:
        motion = 0
        motion_text = "WAIT"
    else:
        if not pir_initialized:
            motion = 0
            pir_initialized = True
        else:
            motion = raw_motion
            motion_text = "YES" if motion == 1 else "NO"

    if light_value < 1800:
        light_state = "BRIGHT"
        rgb_status = "OFF"
        go_to_color((0, 0, 0))
    else:
        light_state = "DARK"
        if pir_ready and motion == 1:
            last_motion_time = time.time()
            rgb_status = "WHITE"
            go_to_color((500, 500, 500))
        else:
            elapsed = time.time() - last_motion_time
            if pir_ready and elapsed < motion_timeout:
                rgb_status = "WHITE"
                go_to_color((500, 500, 500))
            else:
                rgb_status = "DIM BLUE"
                go_to_color((0, 0, 120))

# MANUAL ON
elif system_mode == "MANUAL_ON":
    light_state = "-"
    motion_text = "-"
    rgb_status = "GREEN"
    go_to_color((0, 500, 0))

# MANUAL OFF
elif system_mode == "MANUAL_OFF":
    light_state = "-"
    motion_text = "-"
    rgb_status = "RED"
    go_to_color((500, 0, 0))

update_oled()

# MQTT Publish
if mqtt_client is not None and time.time() - last_mqtt_publish > mqtt_publish_interval:
    try:
        mqtt_client.publish(MQTT_TOPIC_MODE, system_mode)
        mqtt_client.publish(MQTT_TOPIC_LIGHT, str(light_value))
        mqtt_client.publish(MQTT_TOPIC_STATE, light_state)
        mqtt_client.publish(MQTT_TOPIC_MOTION, motion_text)
        mqtt_client.publish(MQTT_TOPIC_RGB, rgb_status)
        last_mqtt_publish = time.time()
        print("MQTT Data Published")
    except OSError:
        print("MQTT Publish Failed")

print("Mode:", system_mode)
print("Light:", light_value)
print("State:", light_state)
print("Motion:", motion_text)
print("RGB:", rgb_status)
print("-----")
time.sleep(1)

```

Conclusion

The ESP32 Smart Room Controller V4 demonstrates a complete practical IoT architecture. It combines embedded programming, sensor-based automation, MQTT communication, remote dashboards, OLED visualization and smart energy principles. The project can be presented as a professional prototype for smart home, smart building, and smart energy applications.

Prepared By:

Ahmad Azroun

Renewable Energy Manager | IoT & Smart Energy Systems Developer